Autómata finito determinista AFD

Imagina un robot con un funcionamiento simple:

“No piensa, no ve, no oye… solo sigue reglas exactas”

Ahora supongamos que nuestro robot en realidad es un autómata y tiene que reconocer si una palabra contiene la letra ‘a’... nuestro robot no ve la palabra completa… lo único que hace es tomar letra por letra y en el momento que encuentre la letra ‘a’ podrá aceptar la palabra al final.

Entonces qué es lo que hace un autómata?

- Es como un Robot muy simple.
- Lee una palabra letra a letra.
- Cambia de modo según las reglas.
- Al final decide si es aceptada la palabra o no.

Lo que también necesitas saber:

- El robot no recuerda… solo sabe su estado actual.
- El robot no puede devolverse.
- El robot no puede adivinar.
- El robot sólo puede seguir reglas exactas.

Entonces en resumen un autómata es:

“una máquina que lee una cadena símbolo por símbolo, cambia de estado según reglas, y al final decide si la acepta o no.”

Ya sabemos que el autómata es un conjunto de elementos:

$$A = (Q, \Sigma, \delta, q0, F)$$

Los elementos de este conjunto son:

- Conjunto de estados (Q) [q0, q1, q2… qn]
- Conjunto de símbolos (Σ) [a, b, c…]
- Conjunto de transiciones (δ) [δ(q0, a) = q1]
- Inicio (q0) [q0]
- Conjunto de estados finales (F) [qx]

Pero no quiero concentrarme de momento en temas matemáticos o demostrativos… vamos a volver a ver términos para definirlos y así dominarlos a la perfección:

Que es un estado

Un estado es el “modo” en el cual se encuentra el autómata en un momento dado, es como la “memoria” del robot.

Para nuestro conjunto se representa con (Q) [Conjunto de estado] … nosotros por lo general los representamos con q0, q1… qn. Y existe para poder diferenciar los estados finales (ESTADOS DE ACEPTACIÓN) de los otros estados que son el inicio o auxiliares.

Ahora para ponernos más técnicos: Un estado es la situación interna en la cual se encuentra el autómata en un instante de procesamiento de una cadena… como el estado es variable para nuestro código se conocerá como “pivote” … que significa “se mueve”.

Para no ponernos tan técnicos: Un estado representa lo que el autómata sabe hasta ese momento.

Un estado puede representar muchas cosas como por ejemplo “una condición cumplida” ó “una condición no cumplida”... puede representar un avance parcial…

Un estado puede representar:

- Una condición.
- Una etapa.
- Una cuenta simplificada.
- Un contexto.

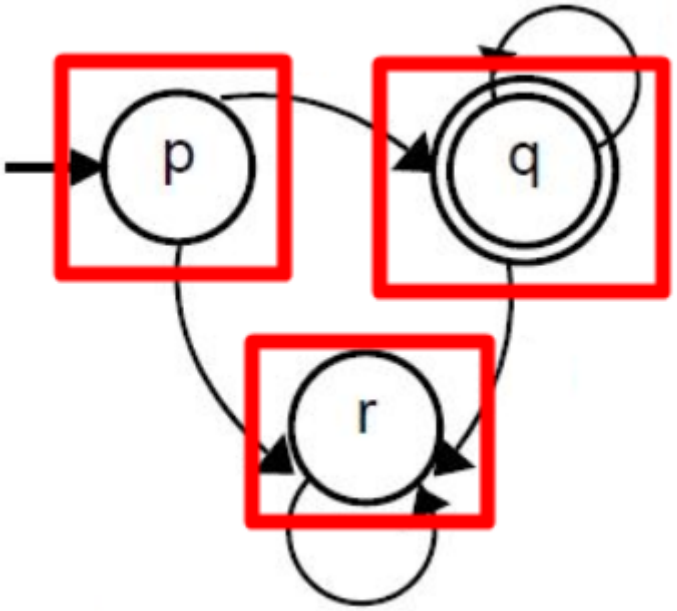


FIG 01: Ejemplo de los 3 estados (p, q, r) de un autómata resaltados en un cuadrado rojo.

Que es una transición

La transición es la regla que hace mover el pivote del autómata, osea… Un autómata lo que hace es leer letra por letra y cambiar su estado según unas reglas exactas pre-definidas.

"La transición es la regla que le dice al autómata como cambiar de estado cuando lee un símbolo"

Para nuestro conjunto se representa con (δ) [δ(q?, x) = q?]... nosotros lo representaremos como que nuestro alfabeto hace mover al pivote de un estado a otro… siendo rigurosos matemáticamente lo vamos a definir con la siguiente función:

$$\delta : Q \times \Sigma \rightarrow Q$$

En donde:

- δ es la toma de un par "estado" y "símbolo".
- Q es el estado inicial.
- Σ es el símbolo.
- Q es el estado destino.

Ejemplo: para δ(q,a)=p lo que se quiere decir es que si el autómata se encuentra en el estado q y entra el símbolo a entonces el pivote se moverá a el estado p.

Entonces lo que tenemos que tener muy claro es que no basta solo con leer la letra para movernos… Tenemos que tener en cuenta el estado actual "recuerden que el pivote siempre empieza por el nodo inicial".

Advertencia

Los caminos para un autómata deben ser únicos osea no se puede escoger entre varios caminos.

Debido a que los caminos deben de tener una sola salida posible significa que nosotros tenemos idealmente que definir todas las combinaciones de caminos posibles… pero me refiero que el autómata debe saber siempre qué hacer.

Para todo $q \in Q$ y para todo $x \in \Sigma$ debe existir una transición.

Por eso podemos ver que los autómatas parecen bucles de descarte, eso se hace para cubrir símbolos que no cambian el modo.

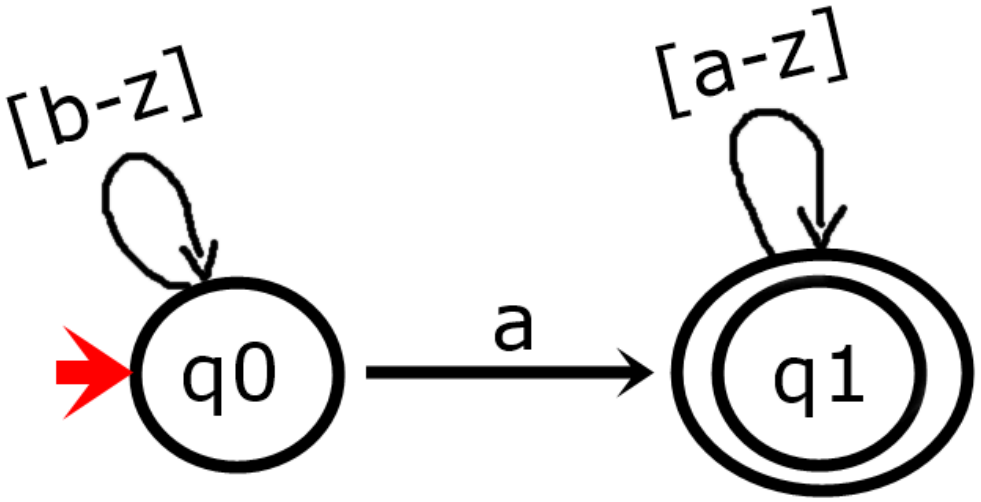
Otra forma de ver las transiciones es:

Si el estado es lo que el autómata sabe hasta el momento pues entonces la transición es una regla que actualiza el conocimiento cuando entra un nuevo símbolo.

En otras palabras:

"El estado es la situación actual y la transición es el cambio de esa situación"

Como se ve la transición en el diagrama:



Como se ve la transición en código:

```
_transitions = _transitions + [
    ("q0", "q1", "a")
]
```

En sí un autómata no interpreta, no reflexiona y no recuerda todo. Lo único que tiene es un estado actual y una entrada actual. La transición le indica cómo reaccionar ante la combinación… La transición es la ley de comportamiento del autómata.

Transiciones Lambda (EXTRA-INFO)

En los Autómatas ε-NFA pueden ocurrir transiciones que ocurren sin consumir un símbolo son llamadas:

"transiciones epsilon"

Pero esto solo es a modo de introducción, de momento solo queremos informar de su existencia pero no vamos a profundizar.

Que es aceptar/rechazar

En pocas palabras:

"Aceptar es cuando un autómata después de leer todos los símbolos de una cadena queda en un estado final"

```
ADF1.pivot in ADF1.F
```

y como es obvio pues rechazar significa que el autómata después de leer todos los símbolos el autómata termina en un estado que no es final. Un autómata no decide durante la lectura, solo al final cuando termina la cadena.

paso a paso:

- Existen unos símbolos: (Σ) [a, ,b, c…]
- Existen Estados: (q) [q0, q1, …, qn]
- Existen transiciones: (δ) [δ(q?, x) = q?]
- Existe una cadena: (w) [x1, x,2, x3… xn]
- Para cada símbolo xi desde el primero hasta el último se aplica la regla de transición (δ) y se actualiza el estado.
- Cuando termina la cadena se mira el estado actual y si el estado pertenece al conjunto de "estados finales" pues entonces se acepta… de lo contrario se rechaza.

```python
def step(self, s):
    for (k,v,n) in self.transitions:
        if k == self.pivot and n == s:
            self.pivot = v
            return True
    return False

def process(self, word):
    self.reset()
    for symbol in word:
        if not self.step(symbol):
            return False
    return self.pivot in self.F
```

Nota: si en algún punto entra un símbolo con una transición no definida pues entonces la cadena debe de ser rechazada inmediatamente.

Aceptar no significa bueno o correcto como en el sentido humano. Simplemente aceptar significa que la cadena cumple con las especificaciones que el autómata representa.

Matemáticamente:

$$\text{Aceptar: } (\delta)(q0,w) \in F$$
$$\text{Rechazar: } (\delta)(q0,w) \text{ not} \in F$$

Procesamiento de Cadenas: cómo ejecutar un autómata su paso a paso.

Un autómata no ve la cadena completa. Solo recibe un símbolo a la vez, desde el primero hasta el último. y en cada paso se cumple lo siguiente:

- Se tiene un estado actual (q?).
- Se lee un símbolo de la cadena (xi).
- Se busca en las funciones de transición (δ) que nuevo estado corresponde para el par actual (q?, xi).
- Se cambia el estado de acuerdo a lo que corresponda a lo que se encontró en la función de transición (δ).
- Se repite el proceso descrito anteriormente hasta que ya no queden más símbolos en la cadena.
- Al final se determina si el estado actual (q?) pertenece al conjunto de estados finales (F).

El autómata no recuerda estados anteriores, ni tampoco recuerda símbolos pasados. Solo mantiene el estado actual como única memoria.

Pseudocódigo:

```
function ejecutar_AFD(cadena):
   estado_actual = q0
   para cada simbolo s en cadena (de izquierda a derecha):
      estado_actual = delta(estado_actual, s)
   fin para
   si estado_actual está en F:
      retornar ACEPTAR
   else:
      retornar RECHAZAR
```

¿Qué pasa con la cadena vacía en una transición?

La cadena vacía es un caso especial, pues se procesa sin leer ningún símbolo… pues entonces:

- si (q0) pertenece a F entonces acepta la cadena vacía.

Ejemplo: para el autómata que contiene la letra ‘a’ (q0) no pertenece a (F) por ello la cadena vacía no es aceptada por el autómata.

Que puede y que no puede hacer un AFD

Ahora experimentamos la diferencia entre una persona bruta y una inteligente… el bruto solo acepta pero el inteligente cuestiona más allá de lo evidente… y ya que estamos preguntando ¿Por qué? pues entonces

Que puede hacer:

Un AFD puede reconocer exactamente los lenguajes y esto incluye los patrones que requieren una cantidad finita de memoria.

Capacidad	***Ejemplo***	***Explicación***
Reconocer cadenas que: ● Empiezan ● Contienen ● Terminan Con un patrón determinado.	cadenas que contengan la letra ‘a’	solamente necesita recordar si vio la letra a y luego cambiar su estado a uno de aceptación. Osea que sus estados son: No he visto la ‘a’ y ya vi la ‘a’
Contar un módulo n	Cadenas que contengan un número par de ‘a’	Un AFD con 2 estados “par” e “impar” no se necesita saber el número exacto solo cambiar de estado a según entra la letra ‘a’
Contar hasta un límite n	Cadenas que contengan 3 veces la letra ‘a’	Una AFD con 5 estados: 0, 1, 2, 3, X A medida que las ‘a’ van entrando mover el estado… En caso de que no exista ‘a’ no se mueve… pero en

		caso de pasarse de 3 pues entonces va a estado muerto.
Verificar la longitud de una cadena	Una cadena contiene menos de 5 caracteres.	estado que cuentan del 1 al 4 y un estado muerto. A medida que van entrando caracteres va cambiando de estados y si se pasa del tope va a el estado muerto.
Reconocer condiciones booleanas en cadenas	Termina con ‘a’ y empieza con ‘b’	Se puede combinar el producto cartesiano de estados.
Reconocimiento de: ● lenguajes. ● complementos. ● uniones. ● Intersecciones.	Dado un AFD con lenguaje L puede operar para lograr el objetivo.	Los regulares son cerrados bajo estas operaciones.

Que no puede hacer:

- Contar hasta un número arbitrario que depende de la entrada.
- Recordar una cantidad no acotada de información.
- Hacer comparaciones en partes lejanas de una cadena.
- Reconocer palíndromos.
- Hacer aritmética general.

Cómo diseñar un AFD desde un problema

Esta es la habilidad que debes de desarrollar y es por lo que fue escrito esto. Para poder realizar el desarrollo de un problema utilizando un AFD tenemos que seguir la siguiente receta:

Paso 0: Comprensión del problema.

- tenemos que identificar cual es el alfabeto (Σ).
- Tenemos que identificar cual es la condición de aceptación (F).
- Preguntarnos qué pasa con la cadena vacía.

Paso 1: Identificación de los estados.

Un estado no es una línea de código ni una variable, es la situación en la que puede encontrarse el autómata… tenemos que buscar “El conjunto de estados de las situaciones posibles.”

La gran pregunta es:

¿Qué situaciones debe de distinguir?

El truco es saber que necesita recordar el autómata a medida que va avanzando el problema. y eso se llama “identificar las posibles situaciones”. Es obvio que todo AFD necesita un inicio y un final… pero tal vez necesite algo en medio y para determinar que hay en el medio piensa:

- ¿Lo que hay hasta ahora cumple todas las condiciones? si es así pues solo hay dos estados… si no es así es porque se necesitan más estados.}
- El último símbolo xn depende del final?
- He alcanzado cierto hito?

Paso 2: Definir las transiciones para cada estado y cada símbolo.

Para todo estado y todo símbolo (x e Σ) se tiene que tomar una decisión para que estado se va. sigue la siguiente regla de oro:

“la transición debe reflejar cómo se actualiza la “INFORMACIÓN RELEVANTE” al leer un nuevo símbolo”

Paso 3: Verificar que sea determinista y completo.

Se dice que un autómata es determinista si: “Para cada estado y símbolo hay solo una transición (si lo definimos)”

Se dice que un autómata es completo si: “Para cada estado y símbolo existe una transición… si puedes completarlo agregando un estado muerto.”

Consejos para diseñar un autómata:

- Empieza por estados que van reflejando el progreso hacia la condición.
- Empieza dibujando un borrador y ve puliendo.
- Completa todas las transiciones y si un símbolo no cambia la información relevante es por que debe de ir al mismo estado.
- Define claramente los estados finales de acuerdo a las condiciones del problema.
- Verifica usando las cadenas que deben ser aceptadas, rechazadas y nunca olvides la cadena vacía.
- Se puede simplificar un autómata “Si dos estados se comportan igual se pueden fusionar.”... eso se llama minimizar.

Código python de un Autómata

$$A = (Q, \Sigma, \delta, q_0, F)$$

```python
class Automaton:
    def __init__(self, states, alphabet, transitions, q0, F):
        self.states = states
        self.alphabet = alphabet
        self.transitions = transitions
        self.q0 = q0
        self.F = F
        self.pivot = q0
```

Ya sabemos que un autómata es un conjunto de 5 elementos:

Q: Es el conjunto de estados.
E: Es el alfabeto.
S: Son las transiciones.
q0: Es el nodo inicial.
F: Es un conjunto de nodos finales.

Para lograr nuestro código necesitamos lograr lo siguiente:

Automaton

+ states
+ alphabet
+ transitions
+ q0
+ F
+ pivot

+ isComplete()
+ isDeterministic()
+ addTransition()
+ reset()
+ step()
+ process()

```python
def isComplete(self):
    for state in self.states:
        for symbol in self.alphabet:
            found = False
            for (origin, _, sym) in self.transitions:
                if origin == state and sym == symbol:
                    found = True
                    break

            if not found:
                return False

    return True
```

¿Qué significa un Autómata completo?

Un autómata se considera completo cuando para cada estado y símbolo de su alfabeto existe por lo menos una transición definida… A eso se le conoce como (Autómata completo o Autómata totalmente definido).

Matemáticamente hablando:

$$\delta : Q \times \Sigma \rightarrow Q$$

```python
def isDeterministic(self):
    for state in self.states:
        for symbol in self.alphabet:
            count = 0
            for (origin, _, sym) in self.transitions:
                if origin == state and sym == symbol:
                    count += 1
                    if count > 1:
                        return False
    return True
```

¿Qué significa que un autómata es determinista?

Para cada par de estado y símbolo existe una transición. Pero el punto clave es que solo puede ser una.

Matemáticamente hablando:

$$\forall\ q \in Q,\ \forall x \in \Sigma : \left| \{ q' \in Q \mid \delta(q,x) = q' \} \right| \leq 1$$

Diferencia entre determinista y completo

Concepto	***Significado***
Completo	Para cada par (estado, símbolo) existe al menos una transición.
Determinista	Para cada par (estado, símbolo) existe una y solo una transición.

```
def addTransition(self, origin, destination, symbol):
    self.transitions.append((origin, destination, symbol))
```

Aunque podemos hacerlo directamente he decidido crear un método que agrega la transición, es solo por documentación debido a que las transiciones entran al momento de instanciar el autómata.

```
def reset(self):
    self.pivot = self.q0
```

Para reiniciar el Autómata basta con mover el pivote al inicio.

```python
def step(self, s):
    for (k,v,n) in self.transitions:
        if k == self.pivot and n == s:
            self.pivot = v
            return True
    return False
```

Dar un paso es uno de los procesos más sencillos y fundamentales de los autómatas, lo único que tenemos que hacer es buscar entre toda la lista de transiciones en donde se moverá el pivote de acuerdo a “es estado actual del pivote y el símbolo”.

```python
def process(self, word):
    self.reset()
    for symbol in word:
        if not self.step(symbol):
            return False
    return self.pivot in self.F
```

El procesar una cadena es el corazón y el porqué de la existencia del autómata… basta solo con recorrer cada símbolo de la cadena apoyados en el método anterior de dar un paso hasta llegar al final… y recordemos que la finalidad de un autómata es determinar si la cadena es válida o no.

Puntos clave de los autómatas

- El autómata solo avanza, nunca retrocede, no existe el deshacer.
- Las transiciones tienen que ser únicas.
- No hay adivinación… Se llama determinista porque todas las reglas tienen que estar definidas.
- La cadena vacía es válida y se procesa en un solo paso.
- La decisión ACEPTAR/RECHAZAR ocurre al final.
- Si en algún paso no hay una transición definida (Error: ADF incompleto) entonces la cadena se rechaza inmediatamente.
- El autómata no tiene memoria… él solo puede guardar su estado actual.
- El autómata no tiene el concepto de error… solo acepta o rechaza.
- El autómata es un modelo computacional sin recursos. No tiene pila, no tiene cinta adicional, no tiene multiplicación.

Ejercicios propuestos

Escribe un autómata sobre el alfabeto [a-z] que acepte cadenas que contengan la ‘a’.

Escribe un autómata sobre el alfabeto [0, 1] que acepte solo números binarios pares.

Escribe un autómata sobre el alfabeto [0, 1] que acepte solo números binarios que empiecen por 1 y terminen en 0.

Escribe un autómata sobre el alfabeto [a, b, c] que acepte cadenas que contengan al menos 2 caracteres ‘b’.

Escribe un autómata sobre el alfabeto [x, y] que acepte cadenas que tengan un número par de caracteres y terminan en ‘yx’.

Escribe un autómata sobre el alfabeto [a, b] que acepte cadenas que no contengan ‘bb’.